

KNN : Pre-reads

1. KNN Introduction

Description:

KNN is an **instance-based learning algorithm** that does not build an explicit model. Instead, it stores the training data and classifies a new observation by comparing it with the 'K' closest points (neighbors) in the training set. It is used for **classification** and **regression** tasks.

Importance:

- Very simple to understand and implement.
 - **Non-parametric** → makes no assumptions about the underlying data distribution.
 - Useful in domains like recommendation systems, image recognition, and anomaly detection.
-

2. Applications and Usage

Description:

- **Classification:** Handwriting recognition, disease prediction, spam detection.
- **Regression:** Predicting house prices or stock trends based on nearby points.
- **Other:** Anomaly detection, recommender systems.

Importance:

- Shows real-world utility of KNN.
 - Highlights when KNN is appropriate: small-to-medium datasets with well-clustered data points.
-

3. Geometric Intuition

Description:

KNN works on the principle that **similar points are close in feature space**. In 2D/3D space, it's easy to visualize how points cluster, and how neighbors determine classification.

Importance:

- Helps in understanding why **feature scaling** is critical (Euclidean distance is sensitive to scale).
- Encourages visualization of data clusters before model selection.

Example: A 2D scatter plot of fruits (apples vs oranges) can show how a new fruit is classified by its nearest neighbors.

4. Distance Metrics

Description:

KNN relies on a **distance metric** to find nearest neighbors. Common metrics:

- **Euclidean distance:** Straight-line distance, default metric.
- **Manhattan distance:** Sum of absolute differences, better for grid-like data.
- **Minkowski distance:** Generalization of Euclidean and Manhattan.
- **Hamming distance:** For categorical features.

Importance:

- Choosing the right metric affects prediction accuracy.
 - Some metrics are sensitive to feature scale, so normalization is often necessary.
-

5. Implementation of KNN from Scratch

Description:

- Compute distance between new point and all training points.

- Sort distances and pick top **K** neighbors.
- Assign class based on majority vote (for classification) or average (for regression).

Importance:

- Reinforces understanding of **algorithm mechanics**.
 - Helps debug and compare with library implementations.
-

6. KNN using Scikit-learn

Description:

- **KNeighborsClassifier** for classification.
- **KNeighborsRegressor** for regression.
- Options: number of neighbors (**n_neighbors**), distance metric (**metric**), and weighting scheme.

Importance:

- Optimized for performance and commonly used in production.
 - Provides easy experimentation with hyperparameters.
-

7. Weighted KNN

Description:

- Assigns **weights based on distance**, giving closer neighbors more influence.
- Weight functions: inverse distance, uniform (all equal), or custom.

Importance:

- Improves performance when neighbors vary in proximity.
 - Useful for imbalanced or non-uniformly distributed datasets.
-

8. Assumptions of KNN

Description:

- Nearby points are likely similar.
- Feature space is meaningful and scaled.
- No irrelevant/noisy features.

Importance:

- Helps guide preprocessing: feature selection, normalization, and outlier handling.
-

9. Time Complexity of KNN

Description:

- Training: $O(1)$ → just storing the data.
- Prediction: $O(n * d)$ → distance computation for n points with d features.

Importance:

- Explains why KNN is slow on large datasets.
 - Suggests need for optimization techniques like **KD-Trees**, **Ball Trees**, or approximate nearest neighbors.
-

10. Bias-Variance Tradeoff

Description:

- Small **k**: low bias, high variance → sensitive to noise.
- Large **k**: high bias, low variance → smoother decision boundary.

Importance:

- Guides hyperparameter tuning.
 - Helps avoid overfitting or underfitting.
-

11. Multi-class Classification using KNN

Description:

- KNN naturally extends to **multi-class problems** using **majority voting** among neighbors.

Importance:

- Essential for real-world datasets that often have multiple categories.
 - Highlights potential issues when classes are imbalanced.
-

12. Outliers

Description:

- Outliers can **distort distance calculations** and lead to wrong predictions.

Importance:

- Encourages preprocessing: outlier detection/removal, robust scaling.
-

13. KNN-based Imputation

Description:

- Missing values replaced using values of **nearest neighbors**.
- Can be used for both numerical and categorical data.

Importance:

- Preserves data patterns without using global averages.
 - Useful for datasets with sparse missing entries.
-

Part 2: Imbalanced Data & KNN

14. What is Imbalanced Data?

Description:

- Datasets where some classes dominate over others.
- Example: Only 10% of customers churn while 90% stay.

Importance:

- Standard metrics like accuracy can be misleading.
 - Minority classes often represent **critical cases**, e.g., fraud detection or churn prediction.
-

15. Imbalanced Data with KNN

Description:

- KNN tends to **predict the majority class** because neighbors are often majority points.

Importance:

- Demonstrates why additional techniques (weighting, resampling) are necessary.
-

16. Why Accuracy is Not Used in Imbalanced Data

Description:

- Accuracy can be high even if minority class is never predicted correctly.
- **Alternative metrics:** Precision, Recall, F1-score, AUC-ROC.

Importance:

- Ensures evaluation captures the model's performance on the minority class.
-

17. Imbalance with Logistic Regression

Description:

- Logistic Regression is also biased toward the majority class if imbalance is not handled.

Importance:

- Confirms that imbalance handling is **algorithm-agnostic**.
-

18. Handling Imbalance Data

Techniques:

- **Weighted Loss:** Penalize misclassification of minority more.
- **Oversampling:** Duplicate or synthesize minority samples.
- **Undersampling:** Remove majority samples.
- **SMOTE:** Generate synthetic minority samples using interpolation.

Importance:

- Crucial for improving minority class detection.
- Helps in building fair and robust models.

19. Weighted Loss

Description:

- Assign a higher loss weight to minority class in model training.

Importance:

- Ensures models focus on rare but important events.
 - Often easier than resampling for large datasets.
-

20. Churn Prediction using Weighted Loss

Description:

- Predicting customer churn using weighted loss to compensate for minority class.

Importance:

- Real-world use-case for telecom, SaaS, or subscription businesses.
-

21. Oversampling

Description:

- Duplicate minority class instances to match majority class size.

Importance:

- Simple and effective for small datasets.
 - Can cause overfitting if naive duplication is used.
-

22. Undersampling

Description:

- Reduce majority class instances to balance the dataset.

Importance:

- Avoids overfitting but may discard important data.
-

23. SMOTE (Synthetic Minority Oversampling Technique)

Description:

- Generates synthetic minority samples along the line segments joining minority instances.
- Reduces overfitting compared to simple oversampling.

Importance:

- Widely used technique in imbalanced datasets.
 - Improves model generalization for minority class.
-

24. Churn Prediction using SMOTE

Description:

- Use SMOTE to balance churn data before model training.
- The model predicts minority churn class more accurately.

Importance:

- Practical application of SMOTE in business analytics.
- Demonstrates how data preprocessing impacts model performance.